

2023-24

Acropolis Institute of Technology and Research, Indore

Submitted To: Ms. Lucky Saxena



Department of CSE

(Artificial Intelligence & Machine
Learning)

Departmental Elective Lab (AL 505)

Submitted By:

Koushal Sharma

0827AL233D05

Class/Year/Sem : AL_T-2/3rd / 5th

[LAB ASSIGNMENT DEPARTMENTAL ELECTIVE LAB (AL-505)]

The objective of a Deep Learning Lab is to provide hands-on experience and practical knowledge of deep learning concepts, tools, and techniques

**ACROPOLIS INSTITUTE OF TECHNOLOGY & RESEARCH,
INDORE**

Department of CSE (Artificial Intelligence & Machine Learning)

CERTIFICATE

This is to certify that the experimental work entered in this journal as per the B. TECH. III year syllabus prescribed by the RGPV was done by Mr. Koushal Sharma, B.TECH III year V semester in the **Departmental Elective Lab (Deep Learning)** Laboratory of this institute during the academic year 2023- 2024.

Signature of the Faculty

ABOUT THE LABORATORY

The Deep Learning Laboratory is a cutting-edge facility designed to provide hands-on experience in deep learning technologies. Equipped with high-performance GPUs and advanced computational tools, the lab enables students and researchers to implement and experiment with deep learning models. Key topics include neural networks, autoencoders, convolutional neural networks (CNNs), recurrent neural networks (RNNs), and generative adversarial networks (GANs).

The lab emphasizes practical learning, offering opportunities to work with tools like TensorFlow, PyTorch, and Keras. Students explore optimization techniques such as Adam and RMSProp while applying their knowledge to real-world problems in computer vision, natural language processing, and speech recognition.

The facility supports research and encourages participation in competitions, fostering teamwork and innovation. By bridging the gap between theoretical learning and practical applications, the Deep Learning Laboratory prepares students for AI-driven careers and contributes to advancements in artificial intelligence and machine learning..

GENERAL INSTRUCTIONS FOR LABORATORY CLASSES

➤ **DO'S**

- ✓ Without Prior permission do not enter into the Laboratory.
- ✓ While entering into the LAB students should wear their ID cards.
- ✓ The Students should come with proper uniform.
- ✓ Students should sign in the LOGIN REGISTER before entering into the laboratory.
- ✓ Students should come with observation and record note book to the laboratory.
- ✓ Students should maintain silence inside the laboratory.
- ✓ After completing the laboratory exercise, make sure to shutdown the system properly.

➤ **DONT'S**

- ✓ Students bringing the bags inside the laboratory.
 - ✓ Students using the computers in an improper way.
 - ✓ Students scribbling on the desk and mishandling the chairs.
 - ✓ Students using mobile phones inside the laboratory.
 - ✓ Students making noise inside the laboratory.
-

SYLLABUS

Course: Departmental Elective AL 505 (B) Deep Learning

Branch/Year/Sem: Artificial Intelligence & Machine Learning / III / V

Module1: : Introduction History of Deep Learning, McCulloch Pitts Neuron, Multilayer Perceptions (MLPs), Representation Power of MLPs, Sigmoid Neurons, Feed Forward Neural Networks, Back propagation, weight initialization methods, Batch Normalization, Representation Learning, GPU implementation, Decomposition – PCA and SVD..

Module2: :Deep Feedforward Neural Networks, Gradient Descent (GD), Momentum Based GD, Nesterov Accelerated GD, Stochastic GD, AdaGrad, Adam, RMSProp, Auto-encoder, Regularization in auto-encoders, Denoising auto-encoders, Sparse auto-encoders, Contractive auto-encoders, Variational auto-encoder, Auto-encoders relationship with PCA and SVD, Dataset augmentation. Denoising auto encoders,

Module3: Introduction to Convolutional neural Networks (CNN) and its architectures, CCN terminologies: ReLu activation function, Stride, padding, pooling, convolutions operations, Convolutional kernels, types of layers: Convolutional, pooling, fully connected, Visualizing CNN, CNN examples: LeNet, AlexNet, ZF-Net, VGGNet, GoogLeNet, ResNet, RCNN etc. Deep Dream, Deep Art. Regularization: Dropout, drop Connect, unit pruning, stochastic pooling, artificial data, injecting noise in input, early stopping, Limit Number of parameters, Weight decay etc.

Module4: :Introduction to Deep Recurrent Neural Networks and its architectures, Backpropagation Through Time (BPTT), Vanishing and Exploding Gradients, Truncated BPTT, Gated Recurrent Units (GRUs), Long Short Term Memory (LSTM), Solving the vanishing gradient problem with LSTMs, Encoding and decoding in RNN network, Attention Mechanism, Attention over images, Hierarchical Attention, Directed Graphical Models. Applications of Deep RNN in Image Processing, Natural Language Processing, Speech recognition, Video Analytics

Module5: Introduction to Deep Generative Models, Restricted Boltzmann Machines (RBMs), Gibbs Sampling for training RBMs, Deep belief networks, Markov Networks, Markov Chains, Auto-regressive Models: NADE, MADE, PixelRNN, Generative Adversarial Networks (GANs), Applications of Deep Learning in Object detection, speech/ image recognition, video analysis, NLP, medical science etc.

HARDWARE AND SOFTWARE REQUIREMENTS:

S. No.	Name of Item	Specification
1	Computer System	Hard Disk min 5 GB RAM: 4 GB / 8 GB Processor: Intel i3 or above

S. No.	Name of Item	Specification
1	Operating system	Window XP or 2000
	Editor	Google Colab

RATIONALE:

The Deep Learning Laboratory provides a practical platform for students to bridge the gap between theoretical concepts and real-world applications of deep learning. It equips learners with the skills to implement, analyze, and optimize deep learning models, enabling them to tackle challenges in domains like computer vision, natural language processing, and data science. By fostering hands-on experience, the laboratory cultivates innovation, critical thinking, and problem-solving abilities, preparing students for AI-driven industries.

PREREQUISITE:-

1. Knowledge of Python programming.
2. Familiarity with basic concepts of machine learning, including supervised and unsupervised learning.
3. Understanding of fundamental mathematics such as linear algebra, calculus, and probability.
4. Basic exposure to neural networks and optimization techniques.

COURSE OBJECTIVES AND OUTCOMES

➤ Course Objectives

1. To introduce students to the implementation of advanced deep learning techniques.
2. To develop the ability to design, train, and evaluate deep neural networks
3. To explore optimization methods for improving model performance.
4. To provide hands-on experience with deep learning frameworks like TensorFlow and PyTorch.
5. To enable students to apply deep learning techniques to solve real-world problems.

➤ Course Outcomes

At the end of the course student will be able to:

1. Design and implement various deep learning models for specific applications.
 2. Optimize model training using advanced techniques such as Adam and RMSProp.
 3. Apply deep learning solutions to fields like computer vision, NLP, and speech recognition.
 4. Analyze and debug deep learning workflows to improve accuracy and efficiency.
 5. Collaborate effectively on AI-driven projects, showcasing teamwork and innovation.
-

Index

S.No	Date of Exp.	Name of the Experiment	Page No.	Date of Submission	Grade & Sign of the Faculty
1		Implementation of Perceptron Algorithm using OR Gate.	9-10		
2		Implementation of Perceptron Algorithm using AND Gate.	11-12		
3		Simplify the binary Log Loss/Cross Entropy Error Function	13-14		
4		Implementation of an Artificial Neural Network (ANN) in Python.	15-16		
5		A simple ANN Model using a placement dataset with layers.	17-20		
6		Implement a Wine Classification model that can classify different types of wines based on their features, and to train the network to minimize classification errors	21-25		
7		Assumes you have a dataset that includes features relevant to loans, such as income, loan amount, credit score, loan term etc., and a binary target variable indicating whether a loan was approved or not.	26-27		
8		a. Implementation of a Convolution Neural Network (CNN) using Tensor Flow/Keras. b. Implementation of LeNet-5 using the TensorFlow Keras API	28-34		
9		Implementing AlexNet , VGGNet, GoogLeNet, ResNet with MNIST Dataset. And Compare their accuracy.	35-41		
10		Design & implement a simple deep learning network for classification of images use MNIST dataset.	42-44		
11		Implement any one of the algorithms VGG to classify objects in objects.	45-46		
12		Design a deep learning network for fine tuning of convolution networks use MNIST dataset.	47-48		
13		Implement Data Augmentation in Deep Learning use any medical image dataset	49-52		

Program Outcome (PO)

The engineering graduate of this institute will demonstrate:

- a) **Apply** knowledge of mathematics, science, computing and engineering fundamentals to computer science engineering problems.
- b) Able to **identify, formulate**, and demonstrate with excellent programming, and problem solving skills.
- c) **Design solutions** for engineering problems including design of experiment and processes to meet desired needs **within reasonable constraints** of manufacturability, sustainability, ecological, intellectual and health and safety considerations.
- d) Propose and develop effective **investigational** solution of complex problems using research methodology; including design of experiment, analysis and interpretation of data, and combination of information to provide suitable conclusion. synthesis
- e) Ability to create, select and use the **modern techniques** and various **tools** to solve engineering problems and to evaluate solutions with an understanding of the limitations.
- f) Ability to acquire knowledge of **contemporary issues** to assess societal, health and safety, legal and cultural issues.
- g) Ability to evaluate the **impact** of engineering solutions on individual as well as organization in a societal and environmental context, and recognize sustainable development, and will be aware of emerging technologies and current professional issues.
- h) Capability to possess leadership and managerial skills, and understand and commit to professional **ethics** and responsibilities.
- i) Ability to demonstrate the team work and **function** effectively as an individual, with an ability to design, develop, test and debug the project, and will be able to work with a multi-disciplinary team.
- j) Ability to **communicate effectively** on engineering problems with the community, such as being able to write effective reports and design documentation.
- k) Flexibility to feel the recognition of the need for, and have the ability to engage in independent and **life- long learning by** professional development and quality enhancement programs in context of technological change.
- l) A **practice** of engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and entrepreneurship

Acropolis Institute of Technology and Research, Indore

Department of CSE (Artificial Intelligence & Machine Learning)

Lab: AL 503 (B) Deep Learning		Group / Title: Implementation of Perceptron Algorithm using OR Gate.	
EVALUATION RECORD		Type/ Lab Session:	
Name	Koushal Sharma	Enrollment No.	0827AL233D05
Performing on		First submission	Second submission
Extra	Regular		
Grade and Remarks by the Tutor			
1. Clarity about the objective of experiment			
2. Clarity about the Outcome			
3. Submitted the work in desired format			
4. Shown capability to solve the problem			
5. Contribution to the team work			

Additional remarks

Grade: Cross the grade.

A	B	C	D	F

Tutor

1 Title: Implementation of Perceptron Algorithm using OR Gate.

2 Program:

```
import numpy as np
#Step 1 : Initialize weights, bias and learning rate
weights = np.array([0,0]) # for two input
bias =0
learning_rate = 0.1
# Step 2 : Define OR + Logic Gate data
X=np.array([[0,0],[0,1],[1,0],[1,1]])
Y=np.array([0,1,1,1])
def activation(z):
    return 1 if z>=0 else 0
#Step 3 : Training the perceptron model
for epoch in range(10):
    for i in range(len(X)):
        z=np.dot(weights, X[i])+bias
        y_pred=activation(z)
        error=Y[i]-y_pred
        weights = weights+ learning_rate*error*X[i]
        bias=bias+learning_rate*error
# Step 5 : Testing the trained perceptron
```

```
for i in range(len(X)):
    z=np.dot(weights, X[i])+bias
    print(f"Input: {X[i]}, Predicted Output: {activation(z)}")
```

3 Output:

```
Input: [0 0], Predicted Output: 0
Input: [0 1], Predicted Output: 1
Input: [1 0], Predicted Output: 1
Input: [1 1], Predicted Output: 1
```

Acropolis Institute of Technology and Research, Indore

Department of CSE (Artificial Intelligence & Machine Learning)

Lab: AL 503 (B) Deep Learning		Group / Title: Implementation of Perceptron Algorithm using AND Gate.	
EVALUATION RECORD		Type/ Lab Session:	
Name	Koushal Sharma	Enrollment No.	0827AL233D05
Performing on		First submission	Second submission
Extra	Regular		
Grade and Remarks by the Tutor			
1. Clarity about the objective of experiment			
2. Clarity about the Outcome			
3. Submitted the work in desired format			
4. Shown capability to solve the problem			
5. Contribution to the team work			

Additional remarks

Grade: Cross the grade.

A	B	C	D	F

Tutor

1 Title: Implementation of Perceptron Algorithm using AND Gate.

2 Program:

```
import numpy as np

# Step 1: Initialize weights, bias, and learning rate
weights = np.array([0, 0]) # for two inputs (X1, X2)
bias = 0
learning_rate = 0.1

# Step 2: Define AND logic gate data (inputs and labels)
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]]) # inputs
y = np.array([0, 0, 0, 1]) # labels (outputs for AND gate)

# Activation function (Step 3)
def activation(z):
    return 1 if z >= 0 else 0

# Step 4: Train the perceptron
for epoch in range(10): # iterate 10 times (or more if needed)
```

```
for i in range(len(X)):
    # Calculate the weighted sum
    z = np.dot(weights, X[i]) + bias
    # Make a prediction
    y_pred = activation(z)
    # Update the weights if prediction is wrong
    error = y[i] - y_pred
    weights = weights + learning_rate * error * X[i]
    bias += learning_rate * error
```

Step 5: Testing the trained perceptron

```
for i in range(len(X)):
    z = np.dot(weights, X[i]) + bias
    print(f"Input: {X[i]}, Predicted Output: {activation(z)}")
```

3 Output:

```
Input: [0 0], Predicted Output: 0
Input: [0 1], Predicted Output: 0
Input: [1 0], Predicted Output: 0
Input: [1 1], Predicted Output: 1
```

Acropolis Institute of Technology and Research, Indore

Department of CSE (Artificial Intelligence & Machine Learning)

Lab: AL 503 (B) Deep Learning		Group / Title: Simplify the binary Log Loss/Cross Entropy Error Function	
EVALUATION RECORD		Type/ Lab Session:	
Name	Koushal Sharma	Enrollment No.	0827AL233D05
Performing on		First submission	Second submission
Extra	Regular		
Grade and Remarks by the Tutor			
1. Clarity about the objective of experiment			
2. Clarity about the Outcome			
3. Submitted the work in desired format			
4. Shown capability to solve the problem			
5. Contribution to the team work			

Additional remarks

Grade: Cross the grade.

A	B	C	D	F

Tutor

1 Title: Simplify the binary Log Loss/Cross Entropy Error Function

2 Program:

```
import numpy as np
def cross_entropy_loss(y_true, y_pred):
    # Ensure predicted values are in the range (0, 1)
    y_pred = np.clip(y_pred, 1e-12, 1. - 1e-12) # Avoid log(0) issue
    # Cross-Entropy Loss formula
    loss = -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))
    return loss

# Example usage:
# y_true are the true labels (0 or 1)
y_true = np.array([1, 0, 1, 1, 0])
# y_pred are the predicted probabilities from the model (between 0 and 1)
y_pred = np.array([0.9, 0.1, 0.8, 0.4, 0.2])

# Calculate Cross Entropy Loss
loss = cross_entropy_loss(y_true, y_pred)
print(f"Cross Entropy Loss: {loss}")
```

3 Output:

Cross Entropy Loss: 0.3146597731636454

Acropolis Institute of Technology and Research, Indore

Department of CSE (Artificial Intelligence & Machine Learning)

Lab: AL 503 (B) Deep Learning		Group / Title: Implementation of Artificial Neural Network (ANN) in Python.	
EVALUATION RECORD		Type/ Lab Session:	
Name	Koushal Sharma	Enrollment No.	0827AL233D05
Performing on		First submission	Second submission
Extra	Regular		
Grade and Remarks by the Tutor			
1. Clarity about the objective of experiment			
2. Clarity about the Outcome			
3. Submitted the work in desired format			
4. Shown capability to solve the problem			
5. Contribution to the team work			

Additional remarks

Grade: Cross the grade.

A	B	C	D	F

Tutor

1 Title: Implementation of an Artificial Neural Network (ANN) in Python.

2 Program:

```
import numpy as np
```

```
class SimpleANN:
```

```
    def __init__(self, input_size, hidden_size, output_size, learning_rate=0.01):
        # Initialize weights and biases
        self.weights1 = np.random.randn(input_size, hidden_size) * 0.01 # Input to hidden
        self.bias1 = np.zeros((1, hidden_size)) # Hidden layer bias
        self.weights2 = np.random.randn(hidden_size, output_size) * 0.01 # Hidden to output
        self.bias2 = np.zeros((1, output_size)) # Output layer bias
        self.learning_rate = learning_rate # Learning rate

    def forward(self, X):
        # Forward pass: compute output
        self.Z1 = np.dot(X, self.weights1) + self.bias1 # Hidden layer linear transformation
        self.A1 = np.maximum(0, self.Z1) # ReLU activation
        self.Z2 = np.dot(self.A1, self.weights2) + self.bias2 # Output layer linear transformation
        self.A2 = 1 / (1 + np.exp(-self.Z2)) # Sigmoid activation
        return self.A2 # Return output
```

```

def backward(self, X, y_true, y_pred):
    # Backward pass: compute gradients and update weights
    m = X.shape[0] # Number of training examples

    # Compute gradients for output layer
    dZ2 = y_pred - y_true # Gradient of loss w.r.t. Z2
    dW2 = np.dot(self.A1.T, dZ2) / m # Gradient of weights for hidden to output
    db2 = np.sum(dZ2, axis=0, keepdims=True) / m # Gradient of bias for output

    # Compute gradients for hidden layer
    dA1 = np.dot(dZ2, self.weights2.T) # Backpropagation to hidden layer
    dZ1 = dA1 * (self.Z1 > 0) # Derivative of ReLU
    dW1 = np.dot(X.T, dZ1) / m # Gradient of weights for input to hidden
    db1 = np.sum(dZ1, axis=0, keepdims=True) / m # Gradient of bias for hidden

    # Update weights and biases
    self.weights1 -= self.learning_rate * dW1
    self.bias1 -= self.learning_rate * db1
    self.weights2 -= self.learning_rate * dW2
    self.bias2 -= self.learning_rate * db2

def train(self, X, y_true, epochs=1000):
    for epoch in range(epochs):
        y_pred = self.forward(X) # Forward pass
        loss = np.mean(-(y_true * np.log(y_pred + 1e-12) + (1 - y_true) * np.log(1 - y_pred + 1e-12))) # Loss calculation
        self.backward(X, y_true, y_pred) # Backward pass
        if epoch % 100 == 0:
            print(f"Epoch {epoch}, Loss: {loss:.4f}")

# Example usage
if __name__ == "__main__":
    # Sample data: X = features, y = target (binary classification)
    X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]]) # AND gate input
    y = np.array([[0], [0], [0], [1]]) # AND gate output

    # Create ANN instance
    ann = SimpleANN(input_size=2, hidden_size=2, output_size=1, learning_rate=0.1)
    ann.train(X, y, epochs=1000) # Train the model

```

Acropolis Institute of Technology and Research, Indore

Department of CSE (Artificial Intelligence & Machine Learning)

Lab: AL 503 (B) Deep Learning		Group / Title: A simple ANN Model using a placement dataset with layers.	
EVALUATION RECORD		Type/ Lab Session:	
Name	Koushal Sharma	Enrollment No.	0827AL233D05
Performing on		First submission	Second submission
Extra	Regular		
Grade and Remarks by the Tutor			
1. Clarity about the objective of experiment			
2. Clarity about the Outcome			
3. Submitted the work in desired format			
4. Shown capability to solve the problem			
5. Contribution to the team work			

Additional remarks

Grade: Cross the grade.

A	B	C	D	F

Tutor

1 Title: A simple ANN Model using a placement dataset with layers.

2 Program:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

# Load the dataset (for example, replace with your actual dataset)
# Assuming the dataset has features like 'cgpa', 'age', 'experience', 'placements' etc.
# Replace this with the actual file path if you're loading a CSV file from Colab
# For demo purposes, we'll create random data resembling a placement dataset

data = pd.DataFrame({
    'cgpa': np.random.uniform(5.0, 10.0, 1000),
    'age': np.random.randint(20, 30, 1000),
    'experience': np.random.randint(0, 5, 1000),
    'placements': np.random.randint(0, 2, 1000) # Binary target variable (0: Not placed, 1: Placed)
})
```

```

# Features (X) and Target (y)
X = data[['cgpa', 'age', 'experience']]
y = data['placements']

# Split data into training and test sets (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Building the ANN model
model = Sequential()

# Input layer and first hidden layer
model.add(Dense(units=10, activation='relu', input_dim=X_train.shape[1]))

# Second hidden layer
model.add(Dense(units=8, activation='relu'))

# Output layer
model.add(Dense(units=1, activation='sigmoid')) # Sigmoid activation for binary classification

# Compiling the ANN
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Training the model
history = model.fit(X_train, y_train, batch_size=32, epochs=50, validation_data=(X_test, y_test))

# Evaluating the model on test data
y_pred = model.predict(X_test)
y_pred = (y_pred > 0.5) # Convert probabilities to binary output (0 or 1)

# Accuracy and confusion matrix
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print(f'Accuracy: {accuracy * 100:.2f}%')
print('Confusion Matrix:')
print(cm)

# Plotting accuracy and loss
import matplotlib.pyplot as plt

# Plot training & validation accuracy values
plt.plot(history.history['accuracy'])

```

```
plt.plot(history.history['val_accuracy'])
plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Test'], loc='upper left')
plt.show()
```

```
# Plot training & validation loss values
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Test'], loc='upper left')
plt.show()
```

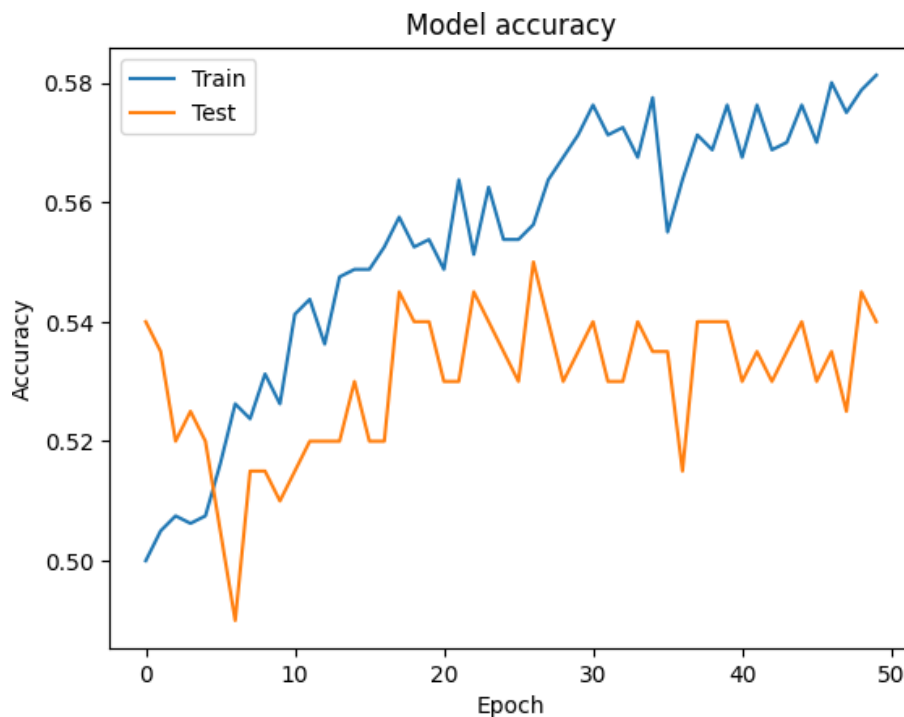
3 Output:

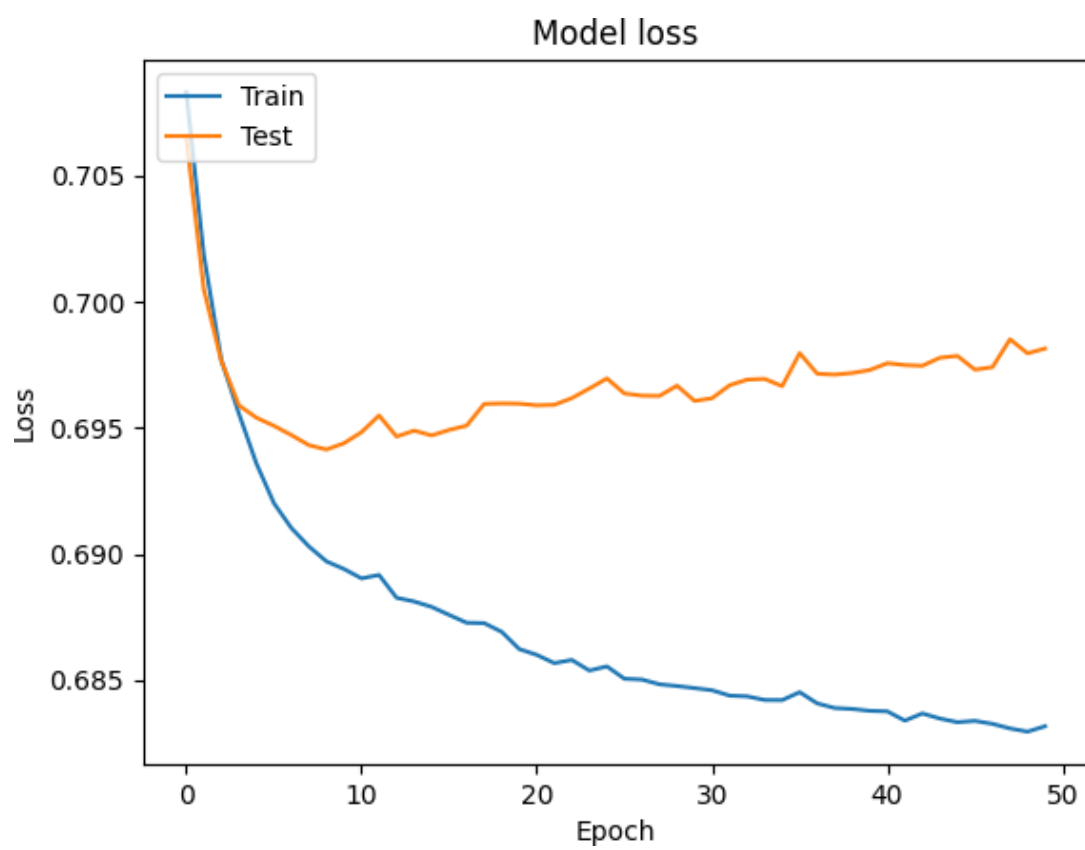
Accuracy: 54.00%

Confusion Matrix:

[[26 75]

[17 82]]





Acropolis Institute of Technology and Research, Indore

Department of CSE (Artificial Intelligence & Machine Learning)

Lab: AL 503 (B) Deep Learning		Group / Title: Implement a Wine Classification model that can classify different types of wines based on their features, and to train the network to minimize classification errors	
EVALUATION RECORD		Type/ Lab Session:	
Name	Koushal Sharma	Enrollment No.	0827AL233D05
Performing on		First submission	Second submission
Extra	Regular		
Grade and Remarks by the Tutor			
1. Clarity about the objective of experiment			
2. Clarity about the Outcome			
3. Submitted the work in desired format			
4. Shown capability to solve the problem			
5. Contribution to the team work			

Additional remarks

Grade: Cross the grade.

A	B	C	D	F

Tutor

- 1 **Title:** Implement a **Wine Classification model** that can classify different types of wines based on their features, and to train the network to minimize classification errors.

2 Program:

```
# Import Required Libraries
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
# white wine dataset
white = pd.read_csv("http://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-white.csv", sep =';')
# red wine dataset
red = pd.read_csv("http://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-red.csv", sep =';')
# Add `type` column to `red` with price one
red['type'] = 1
# Add `type` column to `white` with price zero
white['type'] = 0
# Append `white` to `red`
wines = pd.concat([red, white], ignore_index=True)
```

```

# Import `train_test_split` from `sklearn.model_selection`
from sklearn.model_selection import train_test_split
X = wines.iloc[:, 0:11]
y = np.ravel(wines.type)
# Splitting the data set for training and validating
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size = 0.34, random_state = 45)
# Import `Sequential` from `keras.models`
from keras.models import Sequential
# Import `Dense` from `keras.layers`
from keras.layers import Dense
# Initialize the constructor
model = Sequential()
# Add an input layer
model.add(Dense(12, activation='relu', input_shape=(11, )))
# Add one hidden layer
model.add(Dense(9, activation='relu'))
# Add an output layer
model.add(Dense(1, activation='sigmoid'))
# Model output shape
model.output_shape
# Model summary
model.summary()
# Model config
model.get_config()
# List all weight tensors
model.get_weights()
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
# Training Model
model.fit(X_train, y_train, epochs = 20, batch_size = 1, verbose = 1)
# Predicting the Value
y_pred = model.predict(X_test)
print(y_pred)
from sklearn.metrics import confusion_matrix
# Convert predicted probabilities to binary predictions (0 or 1)
y_pred_binary = (y_pred > 0.5).astype(int)
# Create the confusion matrix
confusion = confusion_matrix(y_test, y_pred_binary)
# Print the confusion matrix
print("Confusion Matrix:")
print(confusion)
# Calculate and print accuracy, precision, and recall
accuracy = (confusion[0, 0] + confusion[1, 1]) / np.sum(confusion)
precision = confusion[1, 1] / (confusion[1, 1] + confusion[0, 1])
recall = confusion[1, 1] / (confusion[1, 1] + confusion[1, 0])
print(f"Accuracy: {accuracy:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")

```

3 Output:

```

/usr/local/lib/python3.10/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
Model: "sequential_2"
┌───────────────────────────────────────────────────────────────────────────────────┐
│ Layer (type)                                     | Output Shape                               │
├───────────────────────────────────────────────────────────────────────────────────┤
│ Param #                                          │                                            │
├───────────────────────────────────────────────────────────────────────────────────┤
│ dense_6 (Dense)                                │ (None, 12)                              │
144 │                                                │                                            │
├───────────────────────────────────────────────────────────────────────────────────┤
│ dense_7 (Dense)                                │ (None, 9)                               │
117 │                                                │                                            │
├───────────────────────────────────────────────────────────────────────────────────┤
│ dense_8 (Dense)                                │ (None, 1)                               │
10 │                                                │                                            │
├───────────────────────────────────────────────────────────────────────────────────┤
Total params: 271 (1.06 KB)
Trainable params: 271 (1.06 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/20
4288/4288 ─────────────────────────────────────────────────────────────────────────── 8s 2ms/step - accuracy:
0.9142 - loss: 0.3637
Epoch 2/20
+4288/4288 ─────────────────────────────────────────────────────────────────────────── 6s 1ms/step - accuracy:
0.9307 - loss: 0.1926
Epoch 3/20
4288/4288 ─────────────────────────────────────────────────────────────────────────── 11s 2ms/step - accuracy:
0.9464 - loss: 0.1584
Epoch 4/20
4288/4288 ─────────────────────────────────────────────────────────────────────────── 11s 2ms/step - accuracy:
0.9596 - loss: 0.1366
Epoch 5/20
4288/4288 ─────────────────────────────────────────────────────────────────────────── 9s 1ms/step - accuracy:
0.9660 - loss: 0.1063
Epoch 6/20
4288/4288 ─────────────────────────────────────────────────────────────────────────── 7s 2ms/step - accuracy:
0.9666 - loss: 0.1005
Epoch 7/20
4288/4288 ─────────────────────────────────────────────────────────────────────────── 10s 2ms/step - accuracy:
0.9742 - loss: 0.0737
Epoch 8/20
4288/4288 ─────────────────────────────────────────────────────────────────────────── 9s 1ms/step - accuracy:
0.9723 - loss: 0.0890
Epoch 9/20

```

```

4288/4288 ————— 10s 1ms/step - accuracy:
0.9673 - loss: 0.0876
Epoch 10/20
4288/4288 ————— 7s 2ms/step - accuracy:
0.9706 - loss: 0.0785
Epoch 11/20
4288/4288 ————— 6s 1ms/step - accuracy:
0.9774 - loss: 0.0778
Epoch 12/20
4288/4288 ————— 11s 2ms/step - accuracy:
0.9733 - loss: 0.0789
Epoch 13/20
4288/4288 ————— 6s 1ms/step - accuracy:
0.9770 - loss: 0.0683
Epoch 14/20
4288/4288 ————— 7s 2ms/step - accuracy:
0.9790 - loss: 0.0749
Epoch 15/20
4288/4288 ————— 10s 2ms/step - accuracy:
0.9711 - loss: 0.0790
Epoch 16/20
4288/4288 ————— 9s 1ms/step - accuracy:
0.9806 - loss: 0.0639
Epoch 17/20
4288/4288 ————— 7s 2ms/step - accuracy:
0.9800 - loss: 0.0631
Epoch 18/20
4288/4288 ————— 9s 1ms/step - accuracy:
0.9785 - loss: 0.0747
Epoch 19/20
4288/4288 ————— 10s 1ms/step - accuracy:
0.9808 - loss: 0.0659
Epoch 20/20
4288/4288 ————— 8s 2ms/step - accuracy:
0.9798 - loss: 0.0582
WARNING:tensorflow:5 out of the last 15 calls to <function
TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_dist
ributed at 0x787d3e9c3eb0> triggered tf.function retracing. Tracing is
expensive and the excessive number of tracings could be due to (1)
creating @tf.function repeatedly in a loop, (2) passing tensors with
different shapes, (3) passing Python objects instead of tensors. For
(1), please define your @tf.function outside of the loop. For (2),
@tf.function has reduce_retracing=True option that can avoid
unnecessary retracing. For (3), please refer to
https://www.tensorflow.org/guide/function#controlling\_retracing and
https://www.tensorflow.org/api\_docs/python/tf/function for more
details.
70/70 ————— 0s 2ms/step
[[0.00303868]
 [0.005675 ]
 [0.00038398]
 ...
 [0.00342992]
 [0.00511771]
 [0.00121939]]
Confusion Matrix:
[[1632    9]

```

```
[ 40 528]]  
Accuracy: 0.98  
Precision: 0.98  
Recall: 0.93
```

Acropolis Institute of Technology and Research, Indore

Department of CSE (Artificial Intelligence & Machine Learning)

Lab: AL 503 (B) Deep Learning		Group / Title: Assumes you have a dataset that includes features relevant to loans, such as income, loan amount, credit score, loan term etc., and a binary target variable indicating whether a loan was approved or not.	
EVALUATION RECORD		Type/ Lab Session:	
Name	Koushal Sharma	Enrollment No.	0827AL233D05
Performing on		First submission	Second submission
Extra	Regular		
Grade and Remarks by the Tutor			
1. Clarity about the objective of experiment			
2. Clarity about the Outcome			
3. Submitted the work in desired format			
4. Shown capability to solve the problem			
5. Contribution to the team work			

Additional remarks

Grade: Cross the grade.

A	B	C	D	F

Tutor

1 Title: Assumes you have a dataset that includes features relevant to loans, such as income, loan amount, credit score, loan term etc., and a binary target variable indicating whether a loan was approved or not.

2 Program:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

```
# Load dataset
df = pd.read_csv('loan_dataset.csv')
```

```
# Define features and target
X = df.drop('LoanApproved', axis=1)
y = df['LoanApproved']
```

```
# Split the data
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train a model
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

# Predict and evaluate
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy:.2f}')
```

3 Output:

Accuracy:0.59

Acropolis Institute of Technology and Research, Indore

Department of CSE (Artificial Intelligence & Machine Learning)

Lab: AL 503 (B) Deep Learning		Group / Title: Implementation of CNN using Tensor Flow/ Keras	
EVALUATION RECORD		Type/ Lab Session:	
Name	Koushal Sharma	Enrollment No.	0827AL233D05
Performing on		First submission	Second submission
Extra	Regular		
Grade and Remarks by the Tutor			
1. Clarity about the objective of experiment			
2. Clarity about the Outcome			
3. Submitted the work in desired format			
4. Shown capability to solve the problem			
5. Contribution to the team work			

Additional remarks

Grade: Cross the grade.

A	B	C	D	F

Tutor

1 Title: Implementation of CNN using Tensor Flow/ Keras.

2 Program:

```
#Imports and Setup
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
#Sample Input Data (5x5 Image with One Channel)
data = np.array([[[[3], [3], [2], [1], [0]],
                  [[0], [0], [1], [3], [1]],
                  [[3], [1], [2], [2], [3]],
                  [[2], [0], [0], [2], [2]],
                  [[2], [0], [0], [0], [1]]]], dtype=np.float32)
#Defining a CNN Model
model = models.Sequential([ layers.Input(shape=(5, 5, 1)), layers.Conv2D(1, (3, 3),
padding='same', strides=(1, 1), activation='relu')])
#Defining the Kernel (Filter)
kernel = np.array([[[[1]], [[0]], [[-1]]],
                  [[[1]], [[0]], [[-1]]],
```

```

[[[1]], [[0]], [[-1]]]])
#Setting the Kernel as the Conv2D Layer's Weights
#used to manually set the weights of the first layer (which is a convolutional layer) in a Keras
model
model.layers[0].set_weights([kernel, np.array([0])])
#Performing Convolution
output = model.predict(data)
#Plotting the Input Image and Output Feature Map
plt.figure(figsize=(8, 4))
# Input Image
plt.subplot(1, 2, 1)
plt.title("Input Image (5x5)")
plt.imshow(data[0, :, :, 0], cmap='gray')
plt.colorbar()
# Output Feature Map
plt.subplot(1, 2, 2)
plt.title("Output Feature Map (5x5)")
plt.imshow(output[0, :, :, 0], cmap='gray')
plt.colorbar()
plt.show()

#Stride and Padding Impact
# Model with stride = 2
model_stride = models.Sequential([
    layers.Input(shape=(5, 5, 1)),
    layers.Conv2D(1, (3, 3), padding='same', strides=(2, 2), activation='relu')
])
model_stride.layers[0].set_weights([kernel, np.array([0])])
# Perform convolution with stride=2
output_stride = model_stride.predict(data)
# Plot input and output
plt.figure(figsize=(8, 4))
# Output with Stride 2
plt.subplot(1, 2, 1)
plt.title("Output with Stride 2 (3x3)")
plt.imshow(output_stride[0, :, :, 0], cmap='gray')
plt.colorbar()
plt.show()

#Pooling Example
# Model with Max Pooling layer
model_pooling = models.Sequential([
    layers.Input(shape=(5, 5, 1)),
    layers.Conv2D(1, (3, 3), padding='same', strides=(1, 1), activation='relu'),
    layers.MaxPooling2D(pool_size=(2, 2))
])
model_pooling.layers[0].set_weights([kernel, np.array([0])])

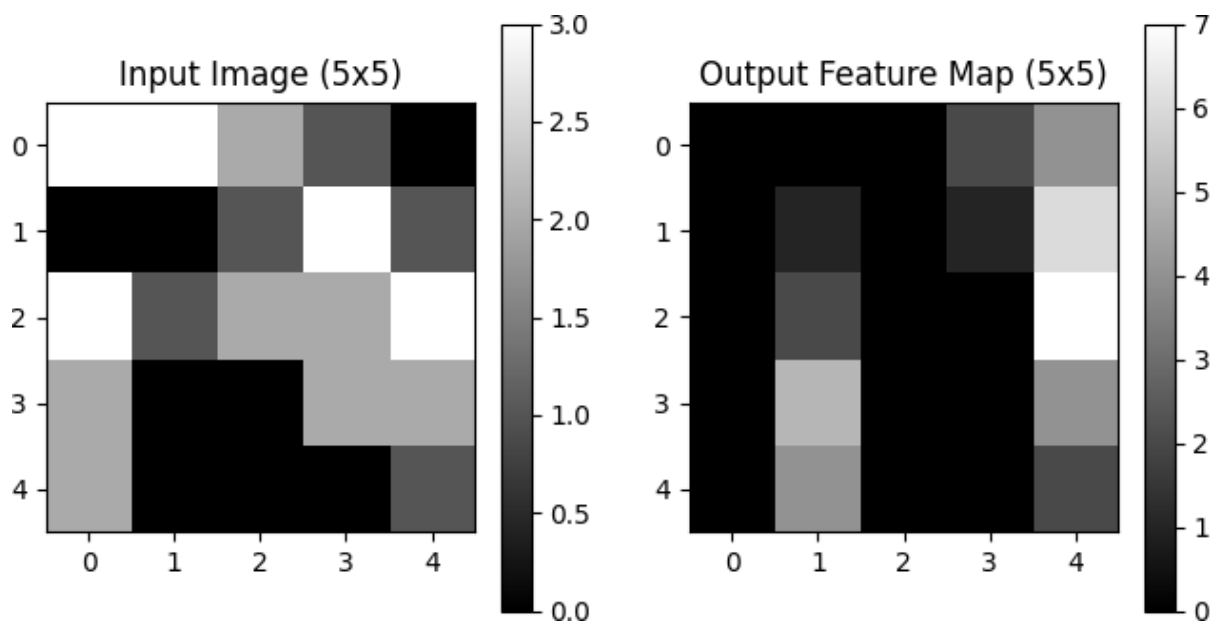
```

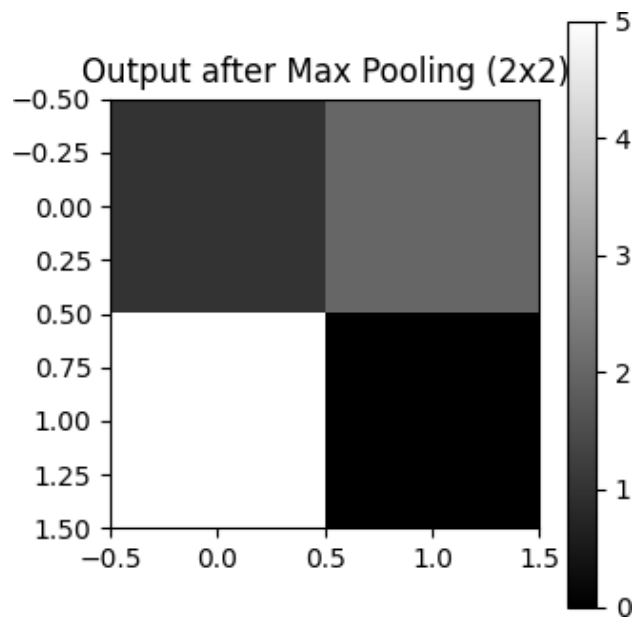
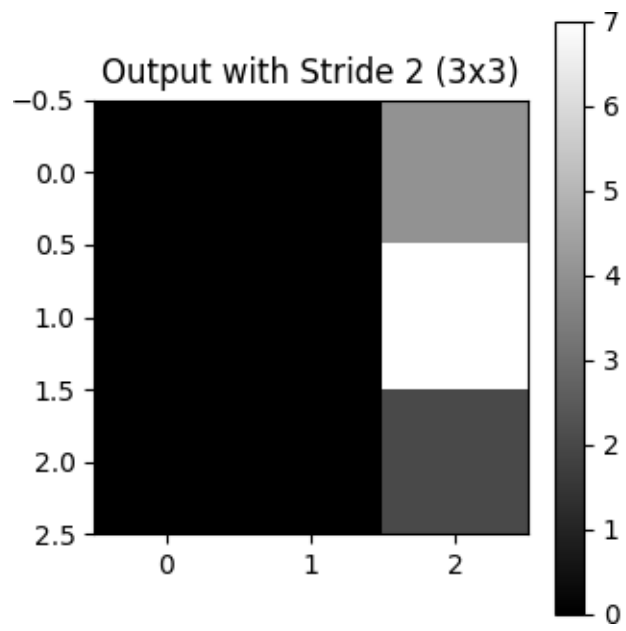
```

# Perform convolution + pooling
output_pooling = model_pooling.predict(data)
# Plot input and output
plt.figure(figsize=(8, 4))
# Output after Pooling
plt.subplot(1, 2, 1)
plt.title("Output after Max Pooling (2x2)")
plt.imshow(output_pooling[0, :, :, 0], cmap='gray')
plt.colorbar()
plt.show()

```

3 Output:





Acropolis Institute of Technology and Research, Indore

Department of CSE (Artificial Intelligence & Machine Learning)

Lab: AL 503 (B) Deep Learning		Group / Title: Implementation of LeNet-5 using the TensorFlow Keras API	
EVALUATION RECORD		Type/ Lab Session:	
Name	Koushal Sharma	Enrollment No.	0827AL233D05
Performing on		First submission	Second submission
Extra	Regular		
Grade and Remarks by the Tutor			
1. Clarity about the objective of experiment			
2. Clarity about the Outcome			
3. Submitted the work in desired format			
4. Shown capability to solve the problem			
5. Contribution to the team work			

Additional remarks

Grade: Cross the grade.

A	B	C	D	F

Tutor

1 Title: Implementation of LeNet-5 using the TensorFlow Keras API

2 Program:

```
import tensorflow as tf
from tensorflow.keras import layers, models
# LeNet-5 Model
def lenet_5():
    model = models.Sequential()

    # Layer 1: Convolutional Layer with 6 filters (5x5), ReLU activation
    model.add(layers.Conv2D(6, (5, 5), activation='relu', input_shape=(32, 32, 1)))

    # Layer 2: Average Pooling (2x2)
    model.add(layers.AveragePooling2D((2, 2)))

    # Layer 3: Convolutional Layer with 16 filters (5x5), ReLU activation
    model.add(layers.Conv2D(16, (5, 5), activation='relu'))

    # Layer 4: Average Pooling (2x2)
    model.add(layers.AveragePooling2D((2, 2)))
```

```

# Layer 5: Flatten the output and add fully connected layers
model.add(layers.Flatten())
model.add(layers.Dense(120, activation='relu')) # Fully connected layer with 120 neurons
model.add(layers.Dense(84, activation='relu')) # Fully connected layer with 84 neurons

# Output layer with 10 neurons (for 10 digit classes)
model.add(layers.Dense(10, activation='softmax'))
return model
# Build the model
model = lenet_5()
# Compile the model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
# Model summary
model.summary()

```

3 Output:

Model: "sequential_4"

Layer (type)	Output Shape
conv2d_13 (Conv2D)	(None, 28, 28, 6)
156 Param #	
average_pooling2d (AveragePooling2D)	(None, 14, 14, 6)
0 Param #	
conv2d_14 (Conv2D)	(None, 10, 10, 16)
2,416 Param #	
average_pooling2d_1 (AveragePooling2D)	(None, 5, 5, 16)
0 Param #	
flatten_4 (Flatten)	(None, 400)
0 Param #	
dense_14 (Dense)	(None, 120)
48,120 Param #	

```

| dense_15 (Dense) | (None, 84) |
10,164 |
|-----|-----|
| dense_16 (Dense) | (None, 10) |
850 |
|-----|-----|

Total params: 61,706 (241.04 KB)
Trainable params: 61,706 (241.04 KB)
Non-trainable params: 0 (0.00 B)

```

Acropolis Institute of Technology and Research, Indore

Department of CSE (Artificial Intelligence & Machine Learning)

Lab: AL 503 (B) Deep Learning		Group / Title: Implementing AlexNet , VGGNet, GoogLeNet, ResNet with MNIST Dataset. And Compare their accuracy.	
EVALUATION RECORD		Type/ Lab Session:	
Name	Koushal Sharma	Enrollment No.	0827AL233D05
Performing on		First submission	Second submission
Extra	Regular		
Grade and Remarks by the Tutor			
1. Clarity about the objective of experiment			
2. Clarity about the Outcome			
3. Submitted the work in desired format			
4. Shown capability to solve the problem			
5. Contribution to the team work			

Additional remarks

Grade: Cross the grade.

A	B	C	D	F

Tutor

1 Title: Implementing AlexNet, VGGNet, GoogLeNet, ResNet with MNIST Dataset. And Compare their accuracy.

2 Program:

```
import tensorflow as tf

from tensorflow.keras import layers, models, optimizers

# Load and preprocess MNIST dataset

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

x_train, x_test = x_train / 255.0, x_test / 255.0 # Normalize

x_train = x_train[..., tf.newaxis] # Add channel dimension

x_test = x_test[..., tf.newaxis]
```

```

# Convert labels to categorical
y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)


# Define AlexNet
def alexnet():
    model = models.Sequential([
        layers.Conv2D(64, (3, 3), activation='relu', input_shape=(28, 28, 1)),
        layers.MaxPooling2D((2, 2)),
        layers.Conv2D(128, (3, 3), activation='relu'),
        layers.MaxPooling2D((2, 2)),
        layers.Flatten(),
        layers.Dense(256, activation='relu'),
        layers.Dropout(0.5),
        layers.Dense(10, activation='softmax')
    ])
    return model


# Define VGGNet
def vggnet():
    model = models.Sequential([
        layers.Conv2D(64, (3, 3), activation='relu', padding='same', input_shape=(28, 28, 1)),
        layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2)),

```

```

layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
layers.MaxPooling2D((2, 2)),
layers.Flatten(),
layers.Dense(256, activation='relu'),
layers.Dropout(0.5),
layers.Dense(10, activation='softmax')
])
return model

```

```

# Define GoogLeNet (simplified for MNIST)

```

```

def googlenet():

```

```

    input_layer = layers.Input(shape=(28, 28, 1))

```

```

    conv1 = layers.Conv2D(64, (3, 3), activation='relu',
padding='same')(input_layer)

```

```

    pool1 = layers.MaxPooling2D((2, 2))(conv1)

```

```

    inception1 = tf.keras.layers.Concatenate()([

```

```

        layers.Conv2D(64, (1, 1), activation='relu', padding='same')(pool1),

```

```

        layers.Conv2D(128, (3, 3), activation='relu', padding='same')(pool1),

```

```

        layers.Conv2D(32, (5, 5), activation='relu', padding='same')(pool1)

```

```

    ])

```

```

    pool2 = layers.MaxPooling2D((2, 2))(inception1)

```

```

    flat = layers.Flatten()(pool2)

```

```

dense = layers.Dense(256, activation='relu')(flat)
output_layer = layers.Dense(10, activation='softmax')(dense)

return models.Model(inputs=input_layer, outputs=output_layer)

# Define ResNet (simplified for MNIST)
def resnet():
    input_layer = layers.Input(shape=(28, 28, 1))

    conv1 = layers.Conv2D(64, (3, 3), activation='relu',
padding='same')(input_layer)

    # Residual Block
    x = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(conv1)
    x = layers.Conv2D(64, (3, 3), activation=None, padding='same')(x)
    res_block = layers.Add()([conv1, x])
    res_block = layers.Activation('relu')(res_block)

    pool = layers.MaxPooling2D((2, 2))(res_block)
    flat = layers.Flatten()(pool)
    dense = layers.Dense(256, activation='relu')(flat)
    output_layer = layers.Dense(10, activation='softmax')(dense)

    return models.Model(inputs=input_layer, outputs=output_layer)

# Compile, Train, and Evaluate

```

```

models_dict = {
    "AlexNet": alexnet(),
    "VGGNet": vggnet(),
    "GoogLeNet": googlenet(),
    "ResNet": resnet()
}

results = {}

for name, model in models_dict.items():

    model.compile(optimizer='adam', loss='categorical_crossentropy',
        metrics=['accuracy'])

    print(f"Training {name}...")

    model.fit(x_train, y_train, epochs=5, batch_size=128, verbose=1)

    _, accuracy = model.evaluate(x_test, y_test, verbose=0)

    results[name] = accuracy

# Compare results

print("\nModel Comparison:")

for name, acc in results.items():

    print(f"{name}: {acc:.4f}")

```

3 Output:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ————— 1s 0us/step
/usr/local/lib/python3.10/dist-packages/keras/src/layers/convolutional/base_conv.py:107:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using
Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
super(). _init_(activity_regularizer=activity_regularizer, **kwargs)
Training AlexNet...
Epoch 1/5
469/469 ————— 115s 242ms/step - accuracy: 0.8581
- loss: 0.4443
Epoch 2/5
469/469 ————— 111s 237ms/step - accuracy: 0.9800
- loss: 0.0668
Epoch 3/5
469/469 ————— 145s 243ms/step - accuracy: 0.9847
- loss: 0.0503
Epoch 4/5
469/469 ————— 139s 237ms/step - accuracy: 0.9888
- loss: 0.0376
Epoch 5/5
469/469 ————— 143s 240ms/step - accuracy: 0.9900
- loss: 0.0325
Training VGGNet...
Epoch 1/5
469/469 ————— 624s 1s/step - accuracy: 0.8688 -
loss: 0.4014
Epoch 2/5
469/469 ————— 681s 1s/step - accuracy: 0.9838 -
loss: 0.0545
Epoch 3/5
469/469 ————— 621s 1s/step - accuracy: 0.9886 -
loss: 0.0377
Epoch 4/5
469/469 ————— 616s 1s/step - accuracy: 0.9918 -
loss: 0.0261
Epoch 5/5
469/469 ————— 599s 1s/step - accuracy: 0.9930 -
loss: 0.0232
Training GoogLeNet...
Epoch 1/5
469/469 ————— 304s 643ms/step - accuracy: 0.9024
- loss: 0.3275
Epoch 2/5
469/469 ————— 327s 655ms/step - accuracy: 0.9870
- loss: 0.0410
Epoch 3/5
469/469 ————— 321s 654ms/step - accuracy: 0.9929
- loss: 0.0224
Epoch 4/5
469/469 ————— 321s 651ms/step - accuracy: 0.9955
- loss: 0.0143
Epoch 5/5
```

```
469/469 ————— 321s 650ms/step - accuracy: 0.9963
- loss: 0.0114
Training ResNet...
Epoch 1/5
469/469 ————— 551s 1s/step - accuracy: 0.9068 -
loss: 0.2967
Epoch 2/5
469/469 ————— 567s 1s/step - accuracy: 0.9893 -
loss: 0.0340
Epoch 3/5
469/469 ————— 561s 1s/step - accuracy: 0.9940 -
loss: 0.0192
Epoch 4/5
469/469 ————— 549s 1s/step - accuracy: 0.9960 -
loss: 0.0122
Epoch 5/5
469/469 ————— 564s 1s/step - accuracy: 0.9965 -
loss: 0.0108

Model Comparison:
AlexNet: 0.9930
VGGNet: 0.9927
GoogLeNet: 0.9883
ResNet: 0.9902
```

Acropolis Institute of Technology and Research, Indore

Department of CSE (Artificial Intelligence & Machine Learning)

Lab: AL 503 (B) Deep Learning		Group / Title: Design & implement a simple deep learning network for classification of images use MNIST dataset.	
EVALUATION RECORD		Type/ Lab Session:	
Name	Koushal Sharma	Enrollment No.	0827AL233D05
Performing on		First submission	Second submission
Extra	Regular		
Grade and Remarks by the Tutor			
1. Clarity about the objective of experiment			
2. Clarity about the Outcome			
3. Submitted the work in desired format			
4. Shown capability to solve the problem			
5. Contribution to the team work			

Additional remarks

Grade: Cross the grade.

A	B	C	D	F

Tutor

1 Title: Design & implement a simple deep learning network for classification of images use MNIST dataset.

2 Program:

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt

# Load MNIST dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# Normalize the data
x_train = x_train / 255.0
x_test = x_test / 255.0

# Reshape the data for the model
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

# Build the model
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
```

```

layers.MaxPooling2D((2, 2)),
layers.Conv2D(64, (3, 3), activation='relu'),
layers.MaxPooling2D((2, 2)),
layers.Flatten(),
layers.Dense(128, activation='relu'),
layers.Dense(10, activation='softmax')
])

# Compile the model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Train the model
history = model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))

# Evaluate the model
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print(f'\nTest accuracy: {test_acc:.4f}')

# Plot training and validation accuracy
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()

```

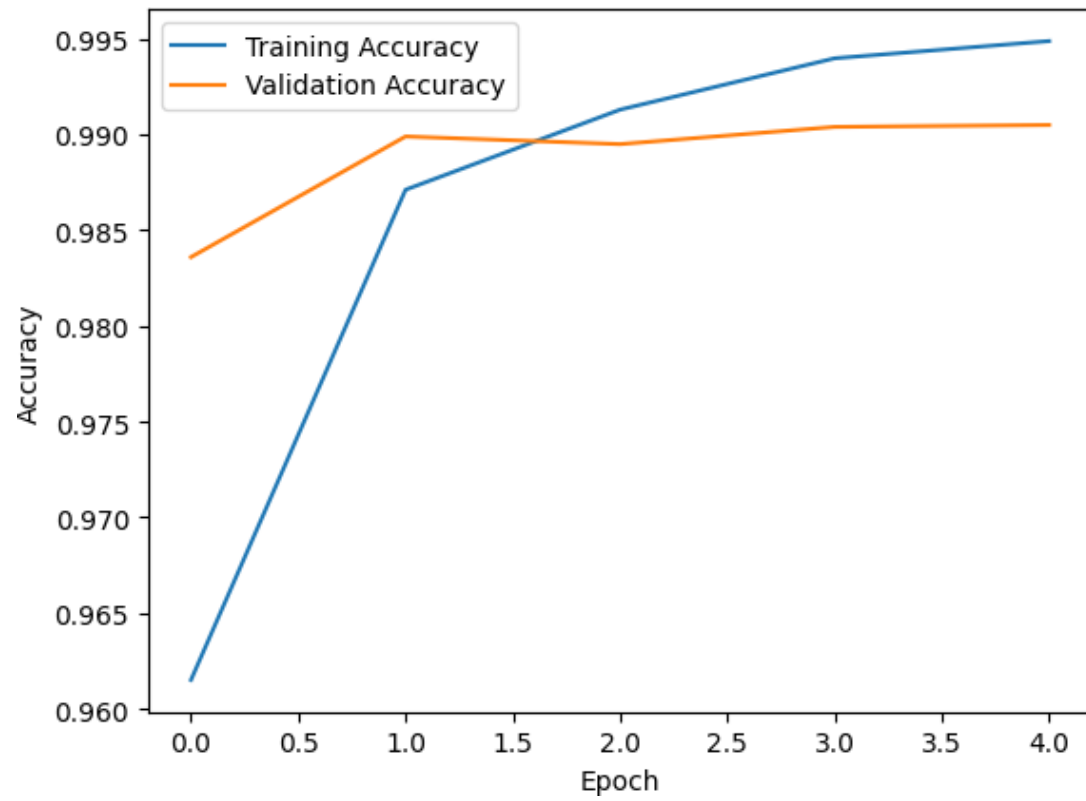
3 Output:

```

Epoch 1/5
1875/1875 _____ 68s 35ms/step - accuracy:
0.9147 - loss: 0.2866 - val_accuracy: 0.9836 - val_loss: 0.0519
Epoch 2/5
1875/1875 _____ 81s 35ms/step - accuracy:
0.9861 - loss: 0.0428 - val_accuracy: 0.9899 - val_loss: 0.0317
Epoch 3/5
1875/1875 _____ 85s 36ms/step - accuracy:
0.9924 - loss: 0.0246 - val_accuracy: 0.9895 - val_loss: 0.0339
Epoch 4/5
1875/1875 _____ 66s 35ms/step - accuracy:
0.9948 - loss: 0.0178 - val_accuracy: 0.9904 - val_loss: 0.0301
Epoch 5/5
1875/1875 _____ 84s 36ms/step - accuracy:
0.9957 - loss: 0.0126 - val_accuracy: 0.9905 - val_loss: 0.0297
313/313 - 3s - 9ms/step - accuracy: 0.9905 - loss: 0.0297

```

Test accuracy: 0.9905



Acropolis Institute of Technology and Research, Indore

Department of CSE (Artificial Intelligence & Machine Learning)

Lab: AL 503 (B) Deep Learning		Group / Title: Implement any one of the algorithms VGG to classify objects in objects.	
EVALUATION RECORD		Type/ Lab Session:	
Name	Koushal Sharma	Enrollment No.	0827AL233D05
Performing on		First submission	Second submission
Extra	Regular		
Grade and Remarks by the Tutor			
1. Clarity about the objective of experiment			
2. Clarity about the Outcome			
3. Submitted the work in desired format			
4. Shown capability to solve the problem			
5. Contribution to the team work			

Additional remarks

Grade: Cross the grade.

A	B	C	D	F

Tutor

1 Title: Implement any one of the algorithms VGG to classify objects in objects

2 .Program:

```
import tensorflow as tf
from tensorflow.keras.applications import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input, decode_predictions
from tensorflow.keras.preprocessing import image
import numpy as np
import matplotlib.pyplot as plt

# Load the pre-trained VGG16 model
model = VGG16(weights='imagenet')

# Load and preprocess the input image
img_path = '1.png' # Replace with your image path
img = image.load_img(img_path, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = preprocess_input(img_array)

# Make predictions
predictions = model.predict(img_array)
```

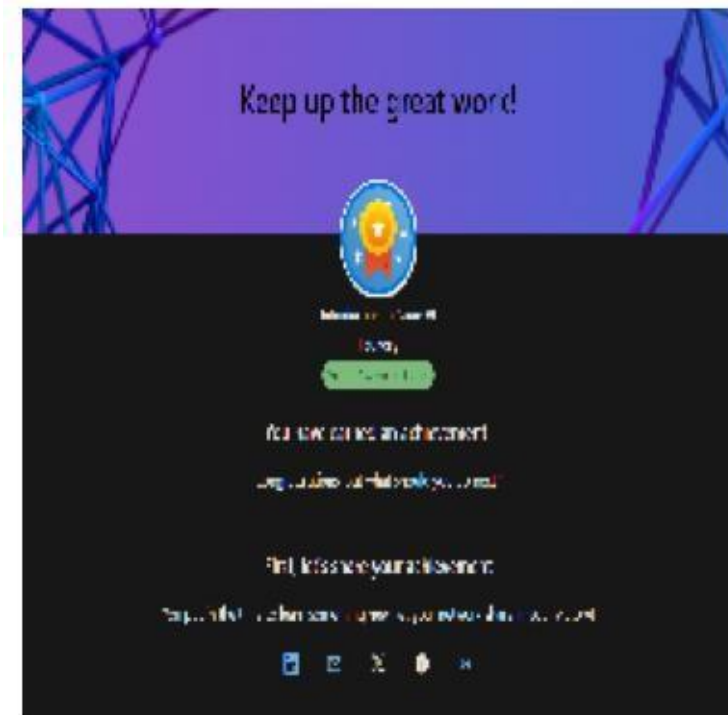
```
# Decode the predictions
decoded_predictions = decode_predictions(predictions, top=3)[0]

# Display the image and predictions
plt.imshow(img)
plt.axis('off')
plt.show()

print("Top-3 Predictions:")
for pred in decoded_predictions:
    print(f'{pred[1]}: {pred[2]*100:.2f}%")
```

3 Output:

1/1 ————— 1s 825ms/step
Downloading data from
https://storage.googleapis.com/download.tensorflow.org/data/imagenet_class_index.json
 35363/35363 ————— 0s 0us/step



Top-3 Predictions:
web_site: 35.76%
bubble: 15.36%
screen: 7.67%

Acropolis Institute of Technology and Research, Indore

Department of CSE (Artificial Intelligence & Machine Learning)

Lab: AL 503 (B) Deep Learning		Group / Title: Design a deep learning network for fine tuning of convolution networks use MNIST dataset.	
EVALUATION RECORD		Type/ Lab Session:	
Name	Koushal Sharma	Enrollment No.	0827AL233D05
Performing on		First submission	Second submission
Extra	Regular		
Grade and Remarks by the Tutor			
1. Clarity about the objective of experiment			
2. Clarity about the Outcome			
3. Submitted the work in desired format			
4. Shown capability to solve the problem			
5. Contribution to the team work			

Additional remarks

Grade: Cross the grade.

A	B	C	D	F

Tutor

1 Title: Design a deep learning network for fine tuning of convolution networks use MNIST dataset.

2 .Program:

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical

# Load and preprocess MNIST dataset
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
train_images = train_images.reshape((60000, 28, 28, 1)).astype('float32') / 255
test_images = test_images.reshape((10000, 28, 28, 1)).astype('float32') / 255

train_labels = to_categorical(train_labels, 10)
test_labels = to_categorical(test_labels, 10)

# Load a pre-trained model (if available) or define a base model
base_model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
```

```

layers.Conv2D(64, (3, 3), activation='relu'),
layers.Flatten(),
layers.Dense(64, activation='relu'),
layers.Dense(10, activation='softmax')
])

# Freeze some layers for fine-tuning
for layer in base_model.layers[:-2]:
    layer.trainable = False

# Compile the model
base_model.compile(optimizer='adam',
                    loss='categorical_crossentropy',
                    metrics=['accuracy'])

# Train the model
history = base_model.fit(train_images, train_labels,
                          epochs=5,
                          batch_size=64,
                          validation_data=(test_images, test_labels))

# Evaluate the model
test_loss, test_acc = base_model.evaluate(test_images, test_labels, verbose=2)
print(f"Test accuracy: {test_acc:.4f}")

```

3 Output:

```

Epoch 1/5
938/938 ————— 24s 24ms/step -accuracy:
0.7464 - loss: 1.2388 - val_accuracy: 0.9284 - val_loss: 0.2728
Epoch 2/5
938/938 ————— 50s 34ms/step -
accuracy: 0.9320 - loss: 0.2523 - val_accuracy: 0.9516 -
val_loss: 0.1740
Epoch 3/5
938/938 ————— 23s 25ms/step -
accuracy: 0.9523 - loss: 0.1720 - val_accuracy: 0.9612 -
val_loss: 0.1393
Epoch 4/5
938/938 ————— 23s 24ms/step -
accuracy: 0.9600 - loss: 0.1414 - val_accuracy: 0.9662 -
val_loss: 0.1206
Epoch 5/5
938/938 ————— 23s 24ms/step -
accuracy: 0.9654 - loss: 0.1210 - val_accuracy: 0.9684 -
val_loss: 0.1086
313/313 - 3s - 10ms/step - accuracy: 0.9684 - loss: 0.1086
Test accuracy: 0.9684

```

Acropolis Institute of Technology and Research, Indore

Department of CSE (Artificial Intelligence & Machine Learning)

Lab: AL 503 (B) Deep Learning		Group / Title: Implement Data Augmentation in Deep Learning use any medical image dataset	
EVALUATION RECORD		Type/ Lab Session:	
Name	Koushal Sharma	Enrollment No.	0827AL233D05
Performing on		First submission	Second submission
Extra	Regular		
Grade and Remarks by the Tutor			
1. Clarity about the objective of experiment			
2. Clarity about the Outcome			
3. Submitted the work in desired format			
4. Shown capability to solve the problem			
5. Contribution to the team work			

Additional remarks

Grade: Cross the grade.

A	B	C	D	F

Tutor

1 Title: Implement Data Augmentation in Deep Learning use any medical image dataset

2 .Program:

```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import os
import pandas as pd
import urllib.request
```

```
# Create necessary directories
sample_dir = './chest_xray/train/NORMAL'
augmented_dir = './augmented_data'
os.makedirs(sample_dir, exist_ok=True)
os.makedirs(augmented_dir, exist_ok=True)
```

```
# Download a sample chest X-ray image
```

```
sample_image_url =
"https://storage.googleapis.com/kagglesdsdata/datasets/17810/23812/chest_xray/test/NORMAL
/IM-0001-0001.jpeg?X-Goog-Algorithm=GOOG4-RSA-SHA256&X-Goog-
Credential=databundle-worker-v2%40kaggle-
161607.iam.gserviceaccount.com%2F20241210%2Fauto%2Fstorage%2Fgoog4_request&X-
```

```

Goog-Date=20241210T215906Z&X-Goog-Expires=345600&X-Goog-
SignedHeaders=host&X-Goog-
Signature=471d4ed08bc62bbf33e3b9dc12a9eca8dca583d45f3e2d691119e3cef8d3938e2061fb0
502a19a1495e4a21bbddb32872dbdbb881a1b7dd04c683baa5d53b2c84dc619c6428a36204ac84
6b01db1a16ad4ad2bb5e7bf08b34972379c48858d1c607c734b073f78b9e3128a8fa0231660ce1e
0829f0860c2c64c5d3b609f3e953f1276c8c6b55123b6955dac128f0f4c76ffb705057561a6c7d24
7265b2ad433266b4db9cd7817c4b4c1d15ad91e1fb4383e905626b5e1a4cbe79d21a16ac77bedcc
ad8af6aa048e972c923e984d22dc557879afd73df95e0edef963b5901ee74a9e3046fac4627b8b99e
2148d8e87333eaf9fc6335f2e0c8c7ea4f7d1e8e08c8" # Updated valid URL
sample_image_path = os.path.join(sample_dir, "sample_image.png")
urllib.request.urlretrieve(sample_image_url, sample_image_path)

# Data Augmentation setup
data_gen = ImageDataGenerator(
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)

# Load and Augment Images
image_data = []
labels = []

for label in os.listdir('./chest_xray/train'):
    class_dir = os.path.join('./chest_xray/train', label)
    if os.path.isdir(class_dir):
        for img_name in os.listdir(class_dir):
            img_path = os.path.join(class_dir, img_name)
            img = tf.keras.preprocessing.image.load_img(img_path, target_size=(224, 224))
            img_array = tf.keras.preprocessing.image.img_to_array(img)
            img_array = img_array.reshape((1,) + img_array.shape)

            # Generate and save augmented images
            prefix = f"{label}_{img_name.split('.')[0]}"
            aug_count = 0
            for batch in data_gen.flow(img_array, batch_size=1, save_to_dir=augmented_dir,
save_prefix=prefix, save_format='jpeg'):
                image_data.append(f"{prefix}_{aug_count}.jpeg")
                labels.append(label)
                aug_count += 1
            if aug_count >= 5: # Limit to 5 augmentations per image
                break

```

```

# Save to CSV
csv_file = 'augmented_data.csv'
augmented_df = pd.DataFrame({'image_name': image_data, 'label': labels})
augmented_df.to_csv(csv_file, index=False)

print(f"Data augmentation completed. CSV file saved as {csv_file}.")

import matplotlib.pyplot as plt
import numpy as np
import tensorflow as tf
import os

# Load and plot some augmented images
augmented_images = os.listdir(augmented_dir)

# Show a grid of augmented images
num_images_to_show = 9
fig, axes = plt.subplots(3, 3, figsize=(10, 10))
fig.suptitle("Sample Augmented Images", fontsize=16)

for i, ax in enumerate(axes.flat):
    if i >= len(augmented_images) or i >= num_images_to_show:
        break
    img_path = os.path.join(augmented_dir, augmented_images[i])
    img = tf.keras.preprocessing.image.load_img(img_path, target_size=(224, 224))
    img_array = tf.keras.preprocessing.image.img_to_array(img).astype(np.uint8)
    ax.imshow(img_array)
    ax.set_title(augmented_images[i])
    ax.axis('off')

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

```

3 Output:

Data augmentation completed. CSV file saved as augmented_data.csv.

Sample Augmented Images

NORMAL_sample_image_0_9452.jpeg



NORMAL_sample_image_0_23.jpeg



NORMAL_sample_image_0_8573.jpeg



NORMAL_sample_image_0_6867.jpeg



NORMAL_sample_image_0_2191.jpeg



NORMAL_sample_image_0_6732.jpeg



NORMAL_sample_image_0_5450.jpeg



NORMAL_sample_image_0_4376.jpeg



NORMAL_sample_image_0_6721.jpeg

